KU LEUVEN

Verslag geavanceerde computerarchitectuur

Sessie 4: Coalesced memory and memory types

Daan Vercammen
Klaas Meersman

ELICT: ELECFS
13 maart 2025
Master in industriële wetenschappen
Academiejaar 2024-2025

1.1 Probleemstelling en doel

In dit labo onderzoeken we de invloed van localiteit bij het oproepen van geheugen, alsook de verschillende types geheugen. Om het effect van localiteit te bestuderen, voeren we bewerkingen uit op een afbeelding, zowel met als zonder herschikking van de te bewerken pixels. Om de verschillende geheugentypes te vergelijken, maken we gebruik van matrixvermenigvuldigingen, waarbij we het algoritme aanpassen aan het gebruikte geheugentype. We analyseren de uitvoeringstijd van de verschillende geheugentypes met bijbehorende algoritmen voor verschillende matrixgroottes.

1.2 Methodologie

We testen de invloed van het gebruikte geheugen en de layout van het geheugen. Voor metingen zullen we telkens de mediaan nemen van verschillende benchmark runs.

1.2.1 Deel 1: Coalesced Memory Access

In dit deel (appendix A) zullen we de rode waarden van een afbeelding inverteren met twee methoden. De eerste methode doorloopt eenvoudigweg de volledige pixelarray en inverteert elke pixel die een veelvoud van 3 is. In de tweede methode herformatteren we eerst de afbeelding zodat alle rode waarden vooraan in de array worden geplaatst. Wanneer we vervolgens de rode waarden inverteren, liggen al deze waarden naast elkaar, wat naar verwachting sneller is dan het oproepen van elke derde waarde, zoals in de eerste methode. We passen dit toe op de afbeelding van de hond. Voor het benchmarken van verschillende afbeeldingsgroottes genereren we vervolgens willekeurige afbeeldingen (die eruitzien als ruis).

1.2.2 Deel 2: Global, shared, and constant memory

In dit deel (appendix B) implementeren we matrixvermenigvuldiging met drie methoden. In methode 1 gebruiken we global memory, wat dient als een basisimplementatie zonder optimalisatie. Methode 2 maakt gebruik van shared memory. Als laatste gebruiken we constant memory, wat geschikt is voor snel geheugen dat tijdens de berekeningen niet meer verandert. We verwachten dat shared memory sneller zal zijn dan constant memory, en dat constant memory op zijn beurt sneller zal zijn dan global memory.

1.3 Resultaten en observaties

1.3.1 Coalesced Memory Access

Coalesced vs. uncoalesced kernel

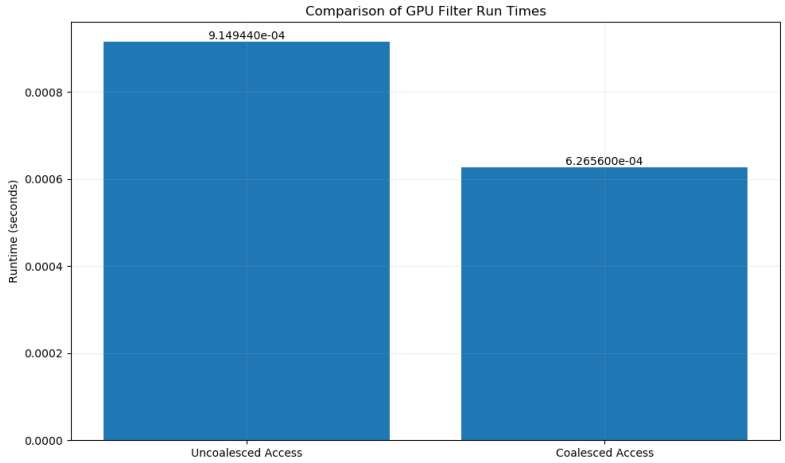
In Figuur 1.1 zien we een duidelijk verschil in uitvoeringstijd. Het algoritme dat werkt met de getransformeerde afbeelding van de hond is ongeveer 30% sneller. Dit komt doordat het ophalen van het geheugen veel efficiënter verloopt wanneer de geheugenlocaties naast elkaar liggen, wat coalesced memory access bevordert.

Matrix input size

In Figuur 1.2 merken we op dat dezelfde tendens ook geldt voor grotere afbeeldingen: naburige geheugenlocaties worden efficiënter opgehaald. Dit is in lijn met onze verwachtingen. Daarnaast is het logisch dat grotere afbeeldingen een langere uitvoeringstijd vereisen.

Grid size

In Figuur 1.3 stellen we opnieuw vast dat coalesced accesses beter presteren. We merken ook op dat zeer kleine grid sizes veel iteraties vereisen, wat de prestaties negatief beïnvloedt. Vanaf een bepaalde grid size (64x64) is het echter niet langer voordelig om nog grotere grid sizes te gebruiken. We vermoeden dat dit ook afhankelijk is van de grootte van de afbeelding.

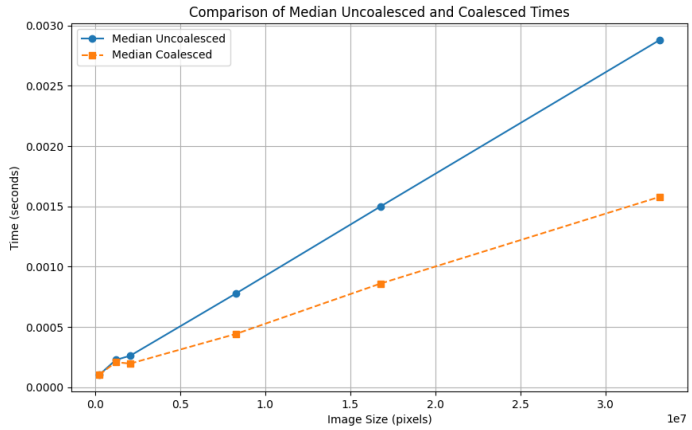


Figuur 1.1: Uncoalesced Memory Access vs. Coalesced Memory Access voor afbeelding hond

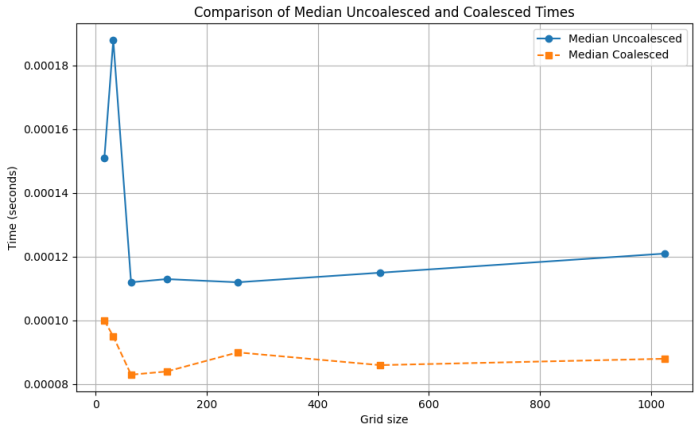
1.3.2 Global, shared, and constant memory

Memory migration

Geheugenmigratie van CPU naar GPU-globaal geheugen is relatief traag vanwege de beperkingen van de PCIe-verbinding, die een lagere bandbreedte heeft vergeleken met interne GPU-verbindingen en ook een grotere afstand moet afleggen. Van GPU-global naar shared memory is veel sneller omdat shared memory zich ook op de GPU bevindt. Het is bovendien direct toegankelijk voor threads binnen een block.



Figuur 1.2: Uncoalesced Memory Access vs. Coalesced Memory Access voor verschillende afbeeldingsgroottes



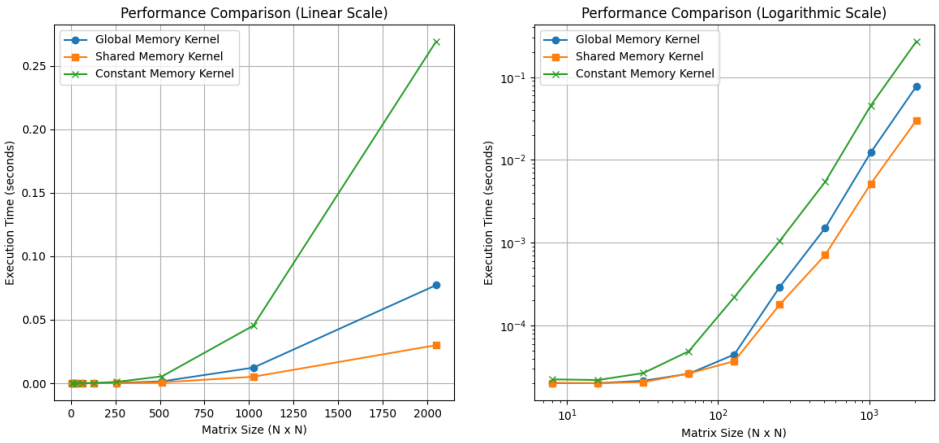
Figuur 1.3: Uncoalesced Memory Access vs. Coalesced Memory Access voor verschillende grid sizes

Global vs. shared vs. constant memory en invloed matrixgrootte

In Figuur 1.4 stellen we vast dat shared memory sneller is dan global memory en dit is ook nog eens sneller dan constant memory. Shared memory is veel sneller dan globaal geheugen omdat het zich op de chip bevindt, waardoor de vertraging veel minder is dan bij ongecachet globaal geheugen. Dit komt doordat shared memory direct toegankelijk is voor threads binnen een threadblock, wat efficiënte samenwerking en minder globale geheugentoegangen mogelijk maakt. Constant memory is het traagst omdat het beperkt is in grootte en vooral geschikt is voor data die onveranderlijk blijft tussen kernel-aanroepen. Hoewel het snel is voor herhaalde toegang, kan het niet concurreren met de snelheid van shared memory, vooral bij complexe berekeningen zoals matrixvermenigvuldiging.

Invloed grid size

De invloed van grid size werd niet getest omdat deze maar een beperkte maximale grootte kunnen hebben, nl. 64 op 64.



Figuur 1.4: Global vs. shared vs. constant memory

1.4 Conclusie

In dit labo hebben we de impact van geheugenlocaliteit en geheugentypes op GPU-prestaties onderzocht. Door coalesced memory access toe te passen, zagen we een aanzienlijke snelheidswinst van ongeveer 30% vergeleken met uncoalesced access, vooral bij grotere afbeeldingen en optimale grid sizes. Shared memory bleek het snelst te zijn voor matrixvermenigvuldiging, gevolgd door global memory en constant memory, vanwege de lagere latentie en hogere bandbreedte. Geheugenmigratie van CPU naar GPU-globaal geheugen was trager dan van GPU-globaal naar shared memory, wat de efficiëntie van on-chip geheugen benadrukt. Deze resultaten tonen aan dat het optimaliseren van geheugentoegang en het kiezen van het juiste geheugentype cruciaal zijn voor het maximaliseren van GPU-prestaties.

A CODE

```cpp
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <algorithm>
#include <math.h>

// #define M 512       // Lenna width
// #define N 512       // Lenna height

#define C 3       // Colors
#define OFFSET 16 // Header length


uint8_t *create_random_image_array(int width, int height, int offset) {
    // Seed the random number generator
    srand(time(NULL));

    // Allocate memory for the image array including the offset
    uint8_t *image_array = (uint8_t *)malloc(width * height * 3 * sizeof(uint8_t) + offset
    );
    if (image_array == NULL) {
        perror("ERROR: Memory allocation failed");
        exit(EXIT_FAILURE);
    }

    // Fill the array with random values (0-255) for RGB components
    for (int i = offset; i < width * height * 3 + offset; i++) {
        image_array[i] = rand() % 256; // Random value between 0 and 255
    }

    // Return the array starting from the offset
    return image_array + offset;
}

void save_image_array(uint8_t *image_array,int M, int N){
    /*
     * Save the data of an (RGB) image as a pixel map.
```

```
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

    // Close the file
    fclose(imageFile);
}



struct info{
    float kernelTime;
    float totalTime;
};

__global__ void REDfilterGPU_Uncoalesced(uint8_t *original, uint8_t *inverted, int size){
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < size){
        if (i % 3 == 0){
            inverted[i] = 255 - original[i];
        }
    }
}


info REDfilterGPUrun_Uncoalesced(uint8_t *image_array, int M, int N){
    // Read the image
    //uint8_t *image_array = get_image_array();
    // Allocate host output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    // Set up grid and block dimensions
    int blockSize = 256;
    int numBlocks = (M * N * C + blockSize - 1) / blockSize;

    // Allocate device memory
    uint8_t *d_original, *d_inverted;
    cudaMalloc((void **)&d_original, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&d_inverted, M * N * C * sizeof(uint8_t));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
```

```
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data to device
    cudaMemcpy(d_original, image_array, M * N * C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernel
    REDfilterGPU_Uncoalesced<<<numBlocks, blockSize>>>(d_original, d_inverted, M * N * C);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy result back to host
    cudaMemcpy(new_image_array, d_inverted, M * N * C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Save the image
    save_image_array(new_image_array,M,N);

    // Free memory
    free(new_image_array);
    cudaFree(d_original);
    cudaFree(d_inverted);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}



void transformImageData(uint8_t *original, uint8_t *transformed, int size){
    for (int i = 0; i < size / 3; ++i){
        // Place red pixels first
        transformed[i] = original[i * 3]; // Red channel

        // Then green pixels
        transformed[size / 3 + i] = original[i * 3 + 1]; // Green channel

```

```
        // Then blue pixels
        transformed[2 * size / 3 + i] = original[i * 3 + 2]; // Blue channel
    }
}

__global__ void redChannelFilter(uint8_t* transformed, uint8_t* inverted, int size) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < size / 3) {
        int redIndex = i; // Red pixels are in the first third of the array

        inverted[redIndex] = 255 - transformed[redIndex]; // Inversion of blue channel
    }
}


void reverseTransformImageData(uint8_t *inverted, uint8_t *finalImage, int size){
    for (int i = 0; i < size / 3; ++i){
        // Place the red pixels back to their original location
        finalImage[i * 3] = inverted[i];

        // Place the green pixels back to their original location
        finalImage[i * 3 + 1] = inverted[size / 3 + i];

        // Place the blue pixels back to their original location
        finalImage[i * 3 + 2] = inverted[2 * size / 3 + i];
    }
}

info REDfilterGPUrun_Coalesced(uint8_t *image_array, int M, int N) {
    // Read the image
    //uint8_t *image_array = get_image_array();


    // Allocate host output and transformed input
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t));
    uint8_t *transformed_input = (uint8_t *)malloc(M * N * C * sizeof(uint8_t));

    // Transform the input data
    transformImageData(image_array, transformed_input, M * N * C);

    // Set up grid and block dimensions
    int blockSize = 256;
    int numBlocks = (M * N * C + blockSize - 1) / blockSize;

    // Allocate device memory
    uint8_t *d_transformed, *d_inverted;
    cudaMalloc((void **)&d_transformed, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&d_inverted, M * N * C * sizeof(uint8_t));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy transformed data to device
    cudaMemcpy(d_transformed, transformed_input, M * N * C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
```

```
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernels
    redChannelFilter<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);
    //greenChannelFilter<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);
    //blueChannelFilter<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy the result back to the host
    cudaMemcpy(transformed_input, d_inverted, M * N * C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    // Reverse transform the data
    reverseTransformImageData(transformed_input, new_image_array, M * N * C);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Save the image
    save_image_array(new_image_array,M,N);

    // Free memory
    free(new_image_array);
    free(transformed_input);
    cudaFree(d_transformed);
    cudaFree(d_inverted);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}


float getMedian(std::vector<float> &v){
    size_t n = v.size() / 2;
    std::nth_element(v.begin(), v.begin() + n, v.end());
    return v[n];
}


int main(void){
    const int NUM_RUNS = 20;
    const int NUM_IMAGE_SIZES = 6;

    // Create vectors to store the times for uncoalesced and coalesced runs for each image
     size
```

```
    std::vector<std::vector<float>> uncoalesced_times(NUM_IMAGE_SIZES, std::vector<float>(NUM_RUNS));
    std::vector<std::vector<float>> coalesced_times(NUM_IMAGE_SIZES, std::vector<float>(NUM_RUNS));

    // Array of different image heights and widths
    int image_dimensions[NUM_IMAGE_SIZES][2] = {
        {512, 512},    // 512x512
        {960, 1280},   // 960x1280
        {1920, 1080},  // 1920x1080
        {3840, 2160},   // 3840x2160
        {4096, 4096},   // 4096x2160
        {7680, 4320}   // 7680x4320
    };

    // For all these sizes, generate a random image and run the filters
    for (int i = 0; i < NUM_IMAGE_SIZES; i++) {
        int M = image_dimensions[i][0];
        int N = image_dimensions[i][1];
        uint8_t *random_image_array = create_random_image_array(M, N, OFFSET);

        // Warm-up run
        REDfilterGPUrun_Uncoalesced(random_image_array, M, N);
        REDfilterGPUrun_Coalesced(random_image_array, M, N);

        // Run the filters NUM_RUNS times and store the runtimes
        for (int run = 0; run < NUM_RUNS; run++) {
            uncoalesced_times[i][run] = REDfilterGPUrun_Uncoalesced(random_image_array, M, N).kernelTime;
            coalesced_times[i][run] = REDfilterGPUrun_Coalesced(random_image_array, M, N).kernelTime;
        }
    }

    // Print the results in a table-like format
    for (int i = 0; i < NUM_IMAGE_SIZES; i++) {
        float median_uncoalesced = getMedian(uncoalesced_times[i]);
        float median_coalesced = getMedian(coalesced_times[i]);

        printf("%d,%.6e,%.6e\n", image_dimensions[i][0] * image_dimensions[i][1], median_uncoalesced, median_coalesced);
    }

    return 0;
}
```

B CODE

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <algorithm>
#include <math.h>

struct info
{
    float kernelTime;
    float totalTime;
};

__global__ void matrixMulKernel(int *A, int *B, int *C, int rowsA, int colsA, int colsB)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rowsA && col < colsB)
    {
        int sum = 0;
        for (int k = 0; k < colsA; ++k)
        {
            sum += A[row * colsA + k] * B[k * colsB + col];
        }
        C[row * colsB + col] = sum;
    }
}


info matrix_multiplication_run(int rowsA, int colsA, int rowsB, int colsB)
{
    // Define small matrices A (FxG), B (GxH), and C (FxH)

    if (colsA != rowsB)
    {
        printf("Matrix dimensions are not compatible for multiplication\n");
        return {0, 0}; // Return early if dimensions are invalid
```

```
    }

    const int F = rowsA;
    const int G = colsA;
    const int H = colsB;

    // Initialize matrices as vectors
    std::vector<int> h_A(F * G);
    std::vector<int> h_B(G * H);
    std::vector<int> h_C(F * H);

    // Random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> distribution(0, 20);

    // Fill matrices A and B with random values
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < G; ++j)
        {
            h_A[i * G + j] = distribution(gen);
        }
    }
    for (int i = 0; i < G; ++i)
    {
        for (int j = 0; j < H; ++j)
        {
            h_B[i * H + j] = distribution(gen);
        }
    }

/*     // Print matrix A
    printf("Matrix A:\n");
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < G; ++j)
        {
            printf("%d\t", h_A[i * G + j]);
        }
        printf("\n");
    }

    // Print matrix B
    printf("Matrix B:\n");
    for (int i = 0; i < G; ++i)
    {
        for (int j = 0; j < H; ++j)
        {
            printf("%d\t", h_B[i * H + j]);
        }
        printf("\n");
    } */

    // Configure grid and block dimensions
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(
        (H + threadsPerBlock.x - 1) / threadsPerBlock.x,
        (F + threadsPerBlock.y - 1) / threadsPerBlock.y);

    // Allocate device memory
```

```
    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, F * G * sizeof(int));
    cudaMalloc(&d_B, G * H * sizeof(int));
    cudaMalloc(&d_C, F * H * sizeof(int));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data from host to device
    cudaMemcpy(d_A, h_A.data(), F * G * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), G * H * sizeof(int), cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch the kernel
    matrixMulKernel<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, F, G, H);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy the result back to the host
    cudaMemcpy(h_C.data(), d_C, F * H * sizeof(int), cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Print the result
/*     printf("Matrix C (Result of A x B):\n");
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < H; ++j)
        {
            printf("%d\t", h_C[i * H + j]);
        }
        printf("\n");
    } */

    // Free device memory
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
```

```
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}



__global__ void matrixMulSharedKernel(int *A, int *B, int *C, int rowsA, int colsA, int
    colsB)
{
    // Define shared memory tiles
    __shared__ int sharedA[16][16];
    __shared__ int sharedB[16][16];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    int sum = 0;

    // Loop over tiles of the matrices
    for (int tile = 0; tile < (colsA + 15) / 16; ++tile)
    {
        // Load elements into shared memory
        if (row < rowsA && (tile * 16 + threadIdx.x) < colsA)
            sharedA[threadIdx.y][threadIdx.x] = A[row * colsA + tile * 16 + threadIdx.x];
        else
            sharedA[threadIdx.y][threadIdx.x] = 0;

        if (col < colsB && (tile * 16 + threadIdx.y) < colsA)
            sharedB[threadIdx.y][threadIdx.x] = B[(tile * 16 + threadIdx.y) * colsB + col
    ];
        else
            sharedB[threadIdx.y][threadIdx.x] = 0;

        __syncthreads(); // Synchronize threads to ensure shared memory is loaded

        // Compute partial sum for the tile
        for (int k = 0; k < 16; ++k)
        {
            sum += sharedA[threadIdx.y][k] * sharedB[k][threadIdx.x];
        }

        __syncthreads(); // Synchronize before loading the next tile
    }

    // Write the result to the output matrix
    if (row < rowsA && col < colsB)
    {
        C[row * colsB + col] = sum;
    }
}


info matrix_multiplication_run_shared(int rowsA, int colsA, int rowsB, int colsB)
{
    // Define small matrices A (FxG), B (GxH), and C (FxH)
    if (colsA != rowsB)
    {
        printf("Matrix dimensions are not compatible for multiplication\n");
        return {0, 0}; // Return early if dimensions are invalid
    }

```

```
    const int F = rowsA;
    const int G = colsA;
    const int H = colsB;

    // Initialize matrices as vectors
    std::vector<int> h_A(F * G);
    std::vector<int> h_B(G * H);
    std::vector<int> h_C(F * H);

    // Random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> distribution(0, 20);

    // Fill matrices A and B with random values
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < G; ++j)
        {
            h_A[i * G + j] = distribution(gen);
        }
    }
    for (int i = 0; i < G; ++i)
    {
        for (int j = 0; j < H; ++j)
        {
            h_B[i * H + j] = distribution(gen);
        }
    }

/*     // Print matrix A
    printf("Matrix A:\n");
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < G; ++j)
        {
            printf("%d\t", h_A[i * G + j]);
        }
        printf("\n");
    }

    // Print matrix B
    printf("Matrix B:\n");
    for (int i = 0; i < G; ++i)
    {
        for (int j = 0; j < H; ++j)
        {
            printf("%d\t", h_B[i * H + j]);
        }
        printf("\n");
    } */

    // Configure grid and block dimensions
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(
        (H + threadsPerBlock.x - 1) / threadsPerBlock.x,
        (F + threadsPerBlock.y - 1) / threadsPerBlock.y);

    cudaFuncSetCacheConfig(matrixMulSharedKernel, cudaFuncCachePreferShared);

    // Allocate device memory
```

```
292     int *d_A, *d_B, *d_C;
293     cudaMalloc(&d_A, F * G * sizeof(int));
294     cudaMalloc(&d_B, G * H * sizeof(int));
295     cudaMalloc(&d_C, F * H * sizeof(int));
296 
297     // Measure total time including data transfer
298     cudaEvent_t start_cuda_total, stop_cuda_total;
299     cudaEventCreate(&start_cuda_total);
300     cudaEventCreate(&stop_cuda_total);
301     cudaDeviceSynchronize();
302     cudaEventRecord(start_cuda_total);
303 
304     // Copy data from host to device
305     cudaMemcpy(d_A, h_A.data(), F * G * sizeof(int), cudaMemcpyHostToDevice);
306     cudaMemcpy(d_B, h_B.data(), G * H * sizeof(int), cudaMemcpyHostToDevice);
307 
308     // Measure kernel execution time separately
309     cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
310     cudaEventCreate(&start_cuda_kernel);
311     cudaEventCreate(&stop_cuda_kernel);
312     cudaDeviceSynchronize();
313     cudaEventRecord(start_cuda_kernel);
314 
315     // Launch the shared memory kernel
316     matrixMulSharedKernel<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, F, G, H);
317 
318     // Measure kernel execution time
319     cudaDeviceSynchronize();
320     cudaEventRecord(stop_cuda_kernel);
321     cudaEventSynchronize(stop_cuda_kernel);
322     float ms_kernel;
323     cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);
324 
325     // Copy the result back to the host
326     cudaMemcpy(h_C.data(), d_C, F * H * sizeof(int), cudaMemcpyDeviceToHost);
327 
328     // Measure total time including data transfer
329     cudaDeviceSynchronize();
330     cudaEventRecord(stop_cuda_total);
331     cudaEventSynchronize(stop_cuda_total);
332     float ms_total;
333     cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);
334 
335 /*     // Print the result
336     printf("Matrix C (Result of A x B using Shared Memory Kernel):\n");
337     for (int i = 0; i < F; ++i)
338     {
339         for (int j = 0; j < H; ++j)
340         {
341             printf("%d\t", h_C[i * H + j]);
342         }
343         printf("\n");
344     } */
345 
346     // Free device memory
347     cudaFree(d_A);
348     cudaFree(d_B);
349     cudaFree(d_C);
350 
351     info info;
352     info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
```

```
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}


// Constant memory matrix
__constant__ int const_B[64 * 64]; // Example: Assuming max matrix size is 256*256
__constant__ int const_A[64 * 64];

__global__ void matrixMulConstMemoryKernel(int *A, int *B, int *C, int rowsA, int colsA,
    int colsB)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rowsA && col < colsB)
    {
        int sum = 0;
        for (int k = 0; k < colsA; ++k)
        {
            sum += const_A[row * colsA + k] * const_B[k * colsB + col];
        }
        C[row * colsB + col] = sum;
    }
}

info matrix_multiplication_run_const(int rowsA, int colsA, int rowsB, int colsB)
{
    // Define small matrices A (FxG), B (GxH), and C (FxH)

    if (colsA != rowsB)
    {
        printf("Matrix dimensions are not compatible for multiplication\n");
        return {0, 0}; // Return early if dimensions are invalid
    }

    const int F = rowsA;
    const int G = colsA;
    const int H = colsB;

    // Initialize matrices as vectors
    std::vector<int> h_A(F * G);
    std::vector<int> h_B(G * H);
    std::vector<int> h_C(F * H);

    // Random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> distribution(0, 20);

    // Fill matrices A and B with random values
    for (int i = 0; i < F; ++i)
    {
        for (int j = 0; j < G; ++j)
        {
            h_A[i * G + j] = distribution(gen);
        }
    }
    for (int i = 0; i < G; ++i)
    {
        for (int j = 0; j < H; ++j)
```

```
        {
            h_B[i * H + j] = distribution(gen);
        }
    }

    // Configure grid and block dimensions
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks(
        (H + threadsPerBlock.x - 1) / threadsPerBlock.x,
        (F + threadsPerBlock.y - 1) / threadsPerBlock.y);

    // Allocate device memory
    int *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, F * G * sizeof(int));
    cudaMalloc(&d_B, G * H * sizeof(int));
    cudaMalloc(&d_C, F * H * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_A, h_A.data(), F * G * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), G * H * sizeof(int), cudaMemcpyHostToDevice);

    // Copy data to constant memory
    cudaMemcpyToSymbol(const_A, h_A.data(), F * G * sizeof(int));
    cudaMemcpyToSymbol(const_B, h_B.data(), G * H * sizeof(int));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch the kernel
    matrixMulConstMemoryKernel<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, F, G, H);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy the result back to the host
    cudaMemcpy(h_C.data(), d_C, F * H * sizeof(int), cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Free device memory
    cudaFree(d_A);
```

```
474     cudaFree(d_B);
475     cudaFree(d_C);
476 
477     info info;
478     info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
479     info.totalTime = ms_total / 1000.0;   // Convert to seconds
480     return info;
481 }
482 
483 
484 
485 
486 float getMedian(std::vector<float> &v)
487 {
488     size_t n = v.size() / 2;
489     std::nth_element(v.begin(), v.begin() + n, v.end());
490     return v[n];
491 }
492 
493 int main(void)
494 {
495     //make bigger and bigger matrix multiplication and get the median of the kernel time
        and total time
496     //print # elements, kernel time, total time
497 
498     const int MAX_SIZE = 2500;
499     const int numberOfReruns = 10;
500     std::vector<std::vector<float>> totalTimesMatrix_global;
501     std::vector<std::vector<float>> kernelTimesMatrix_global;
502     std::vector<std::vector<float>> totalTimesMatrix_shared;
503     std::vector<std::vector<float>> kernelTimesMatrix_shared;
504     std::vector<std::vector<float>> totalTimesMatrix_const;
505     std::vector<std::vector<float>> kernelTimesMatrix_const;
506 
507     for (int i = 8; i <= MAX_SIZE; i *= 2)
508     {
509         std::vector<float> totalTimes_global;
510         std::vector<float> kernelTimes_global;
511         std::vector<float> totalTimes_shared;
512         std::vector<float> kernelTimes_shared;
513         std::vector<float> totalTimes_const;
514         std::vector<float> kernelTimes_const;
515 
516         for (int j = 0; j < numberOfReruns; j++)
517         {
518             info info_global = matrix_multiplication_run(i, i, i, i);
519             info info_shared = matrix_multiplication_run_shared(i, i, i, i);
520             info info_const = matrix_multiplication_run_const(i, i, i, i);
521 
522             totalTimes_global.push_back(info_global.totalTime);
523             kernelTimes_global.push_back(info_global.kernelTime);
524             totalTimes_shared.push_back(info_shared.totalTime);
525             kernelTimes_shared.push_back(info_shared.kernelTime);
526             totalTimes_const.push_back(info_const.totalTime);
527             kernelTimes_const.push_back(info_const.kernelTime);
528         }
529 
530         totalTimesMatrix_global.push_back(totalTimes_global);
531         kernelTimesMatrix_global.push_back(kernelTimes_global);
532         totalTimesMatrix_shared.push_back(totalTimes_shared);
533         kernelTimesMatrix_shared.push_back(kernelTimes_shared);
```

```
534         totalTimesMatrix_const.push_back(totalTimes_const);
535         kernelTimesMatrix_const.push_back(kernelTimes_const);
536     }
537 
538     for (int i = 0; i < totalTimesMatrix_global.size(); i++)
539     {
540         printf("%d, %.6e, %.6e, %.6e\n", (int)pow(2, 3 + i),
541         getMedian(kernelTimesMatrix_global[i]),
542         getMedian(kernelTimesMatrix_shared[i]),
543         getMedian(kernelTimesMatrix_const[i]));
544     }
545 
546 
547     return 0;
548 }
```

REFERENTIELIJST

[1] *CUDA: Determining threads per block, blocks per grid*. Stack Overflow. Geraardpleegd op 17 februari 2025. 2010. URL: https://stackoverflow.com/questions/4391162/cuda-determining-threads-per-block-blocks-per-grid.

[2] NVIDIA Corporation. *CUDA C++ Programming Guide*. Geraadpleegd op 13 februari 2025. NVIDIA. verschillend. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[3] Alex Minnaar. *Grid-stride loops*. Geraadpleegd op 3 maart 2025. Aug 2019. URL: https://alexminnaar.com/2019/08/02/grid-stride-loops.html.